

Empirical Software Engineering: A gentle introduction

Roberto Verdecchia
University of Florence



UNIVERSITÀ
DEGLI STUDI
FIRENZE

DINFO

DIPARTIMENTO DI
INGEGNERIA DELL'INFORMAZIONE



**Software
Technologies
Laboratory**



This is an interactive lecture!
I will ask you questions, and
you should do the same!

Software Repository mining: What is it



Repository mining: What is it

- Collect **software-related artefacts** from one or more software repositories
- Usually, but not necessarily, based on **open-source**, e.g., GitHub
- Based on a **set of project of interest**
- Can consider **one commits** or **whole commit histories**
- **Data is extracted** from the collected artefacts, e.g., source code analysis and or manual labeling
- **Statistical tests** are run to draw conclusions

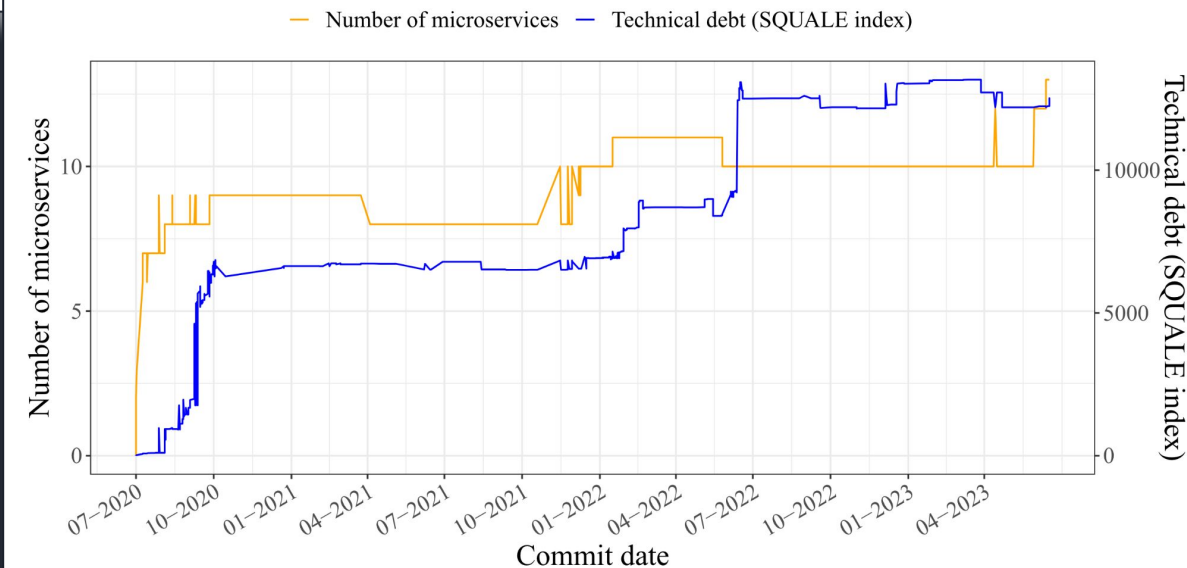
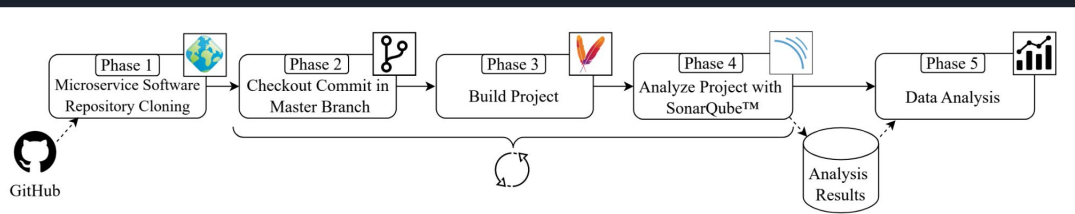


Collection of software artefacts

- Depending on the analysis, **software artefacts** could be:
 - Source code (e.g., tests)
 - Commit history (e.g., commit time, message, ...)
 - Documentation (e.g., README files)
 - Images
 - ...
- For repository **evolution analyses**, usually:
 - **Select a repository**
 - **Check out each repository**
 - **Analyze commit and save data**
 - **Pass to next commit / repository**

```
31 def __init__(self, settings):
32     self.file = None
33     self.fingerprints = set()
34     self.logdupes = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, 'repo'), 'w')
39         self.file.seek(0)
40         self.fingerprints.update(os.listdir(path))
41
42 @classmethod
43 def from_settings(cls, settings):
44     debug = settings.getbool('debug', True)
45     return cls(job_dir(settings), debug)
46
47 def request_seen(self, request):
48     fp = self.request_fingerprint(request)
49     if fp in self.fingerprints:
50         return True
51     self.fingerprints.add(fp)
52     if self.file:
53         self.file.write(fp + os.linesep)
54
55 def request_fingerprint(self, request):
56     return request_fingerprint(request)
```

An example of system evolution analysis



Technical Debt in Microservices: A Mixed-Method Case Study

Roberto Verdecchia^[0000-0001-9206-6637], Kevin Maggi^{*},
Leonardo Scommegna^[0000-0002-7293-0210], and Enrico Vicario^[0000-0002-4983-4386]

Department of Information Engineering, University of Florence, Italy
roberto.verdecchia@unifi.it,
kevin.maggi@edu.unifi.it, leonardo.scommegna@unifi.it, enrico.vicario@unifi.it

Abstract. *Background:* Despite the rising interest of both academia and industry in microservice-based architectures and technical debt, the landscape remains uncharted when it comes to exploring the technical debt evolution in software systems built on this architecture. *Aims:* This study aims to unravel how technical debt evolves in software-intensive systems that utilize microservice architecture, focusing on (i) the patterns of its evolution, and (ii) the correlation between technical debt and the number of microservices. *Method:* We employ a mixed-method case study on an application with 13 microservices, 977 commits, and 38k lines of code. Our approach combines repository mining, automated code analysis, and manual inspection. The findings are discussed with the lead developer in a semi-structured interview, followed by a reflective thematic analysis. *Results:* Despite periods of no TD growth, TD generally increases over time. TD variations can occur irrespective of microservice count or commit activity. TD and microservice numbers are often correlated. Adding or removing a microservice impacts TD similarly, regardless of existing microservice count. *Conclusions:* Developers must be cautious about the potential technical debt they might introduce, irrespective of the development activity conducted or the number of microservices involved. Maintaining steady technical debt during prolonged periods of time is possible, but growth, particularly during innovative phases, may be unavoidable. While monitoring technical debt is the key to start managing it, technical debt code analysis tools must be used wisely, as their output always necessitates also a qualitative system understanding to gain the complete picture.

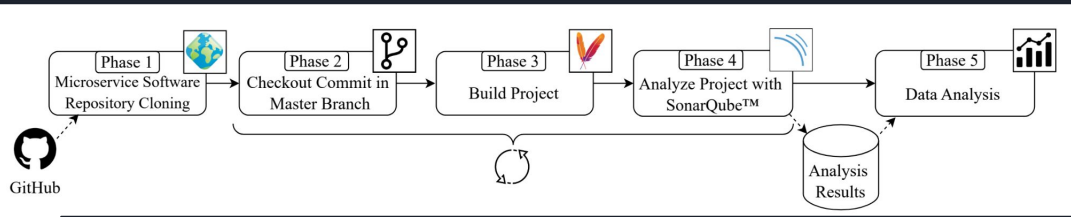
Keywords: Technical Debt · Microservices · Software Evolution

1 Introduction

As companies seek to take advantage of their many benefits, microservice-based architectures are becoming more and more adopted. As often referenced, the microservice architecture style offers several advantages, such as scalability, flexibility, and the ability to develop and deploy individual components independently [11]. Albeit the many benefits microservice-based systems offer, the architectural style also comes with its own set of challenges, including increased complexity, the need for effective management of eventual consistency, and additional effort required for integration and system testing.

^{*} The first two authors contributed equally to this work.

An example of system evolution analysis



Technical Debt in Microservices: A Mixed-Method Case Study

Roberto Verdecchia^[0000-0001-9206-6637], Kevin Maggi^{*},
Leonardo Scommegna^[0000-0002-7293-0210], and Enrico Vicario^[0000-0002-4983-4386]

Department of Information Engineering, University of Florence, Italy
roberto.verdecchia@unifi.it,
kevin.maggi@edu.unifi.it, leonardo.scommegna@unifi.it, enrico.vicario@unifi.it

Abstract. *Background:* Despite the rising interest of both academia and industry in microservice-based architectures and technical debt, the landscape remains uncharted when it comes to exploring the technical debt evolution in software systems built on this architecture. *Aims:* This study aims to unravel how technical debt evolves in software-intensive systems that utilize microservice architecture, focusing on (i) the patterns of its evolution, and (ii) the correlation between technical debt and the number of microservices. *Method:* We employ a mixed-method case study on an application with 13 microservices, 977 commits, and 38k lines of code. Our approach combines repository mining, automated code analysis, and manual inspection. The findings are discussed with the lead developer in a semi-structured interview, followed by a reflective thematic analysis. *Results:* Despite periods of no TD growth, TD generally increases over time. TD variations can occur irrespective of microservice count or commit activity. TD and microservice numbers are often correlated. Adding or removing a microservice count.

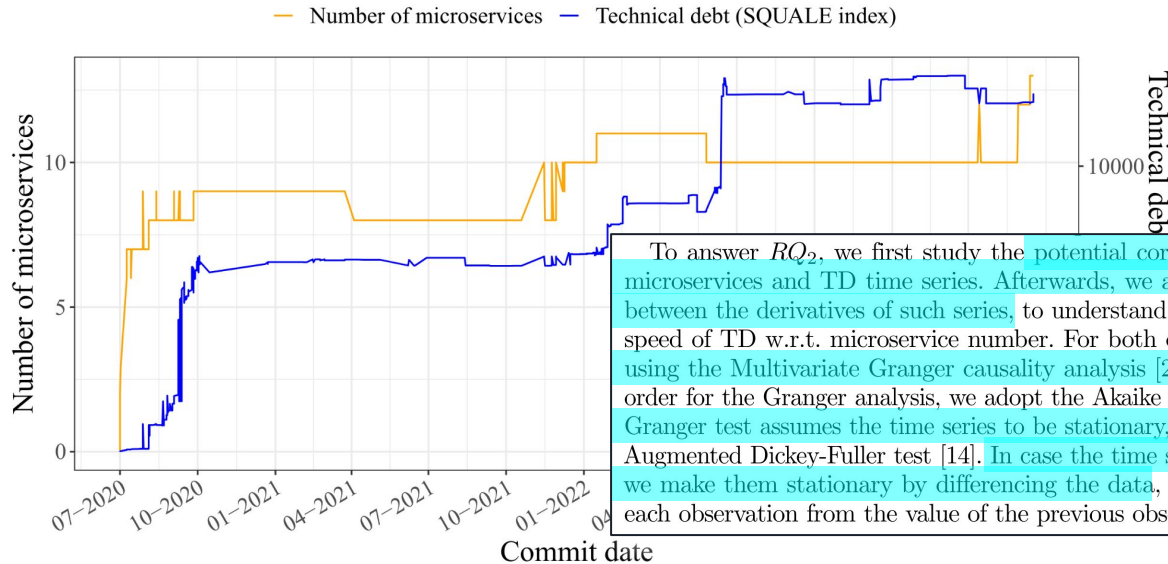
Conclusions: Initial technical debt they might have conducted or the number of technical debt during prolonged periods of innovative phases, may be the key to start managing it, wisely, as their output always gain the complete picture.

Software Evolution

by benefits, microservice-based architectures often referenced, the microservice scalability, flexibility, and the ability to evolve independently [11]. Albeit the many advantages, this architectural style also comes with its own

set of challenges, including increased complexity, the need for effective management of eventual consistency, and additional effort required for integration and system testing.

^{*} The first two authors contributed equally to this work.



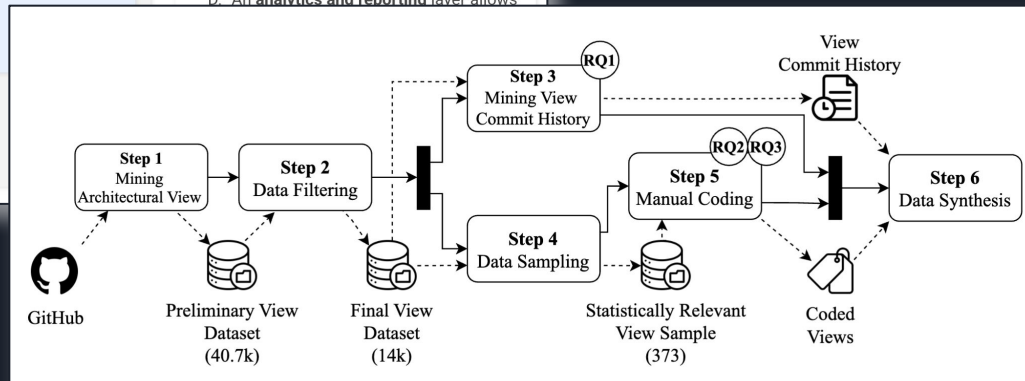
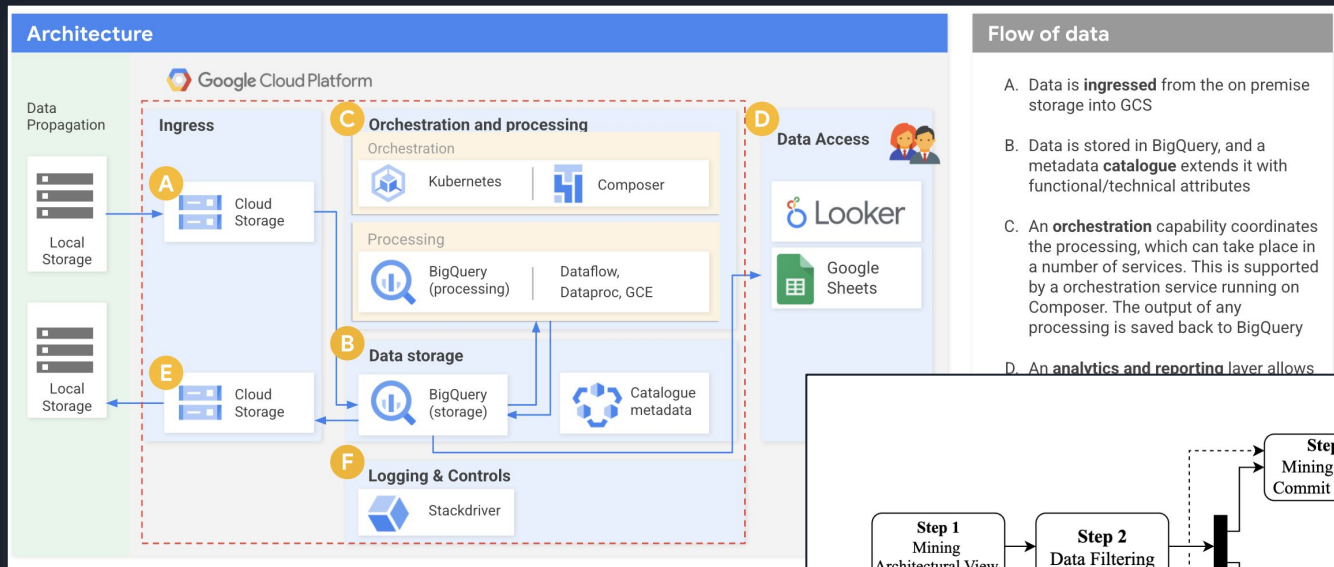
To answer RQ_2 , we first study the potential correlation between the number of microservices and TD time series. Afterwards, we analyze the potential correlation between the derivatives of such series, to understand the relation between the growth speed of TD w.r.t. microservice number. For both cases, we test the correlation by using the Multivariate Granger causality analysis [21]. To calculate the optimal lag order for the Granger analysis, we adopt the Akaike Information Criterion [1]. As the Granger test assumes the time series to be stationary, we test such assumption via the Augmented Dickey-Fuller test [14]. In case the time series result to be non-stationary, we make them stationary by differencing the data, i.e., by subtracting the value of each observation from the value of the previous observation in the time series.

What software artefacts
were collected to conduct
the analysis?

Collection of software artefacts: More than just source code

- **Architectural viewpoint:** A (usually informal) **language to describe a software architecture**. By considering a software product as a building, a viewpoint could be “plumbing blueprint” or “electric wiring blueprint”. Usually, **each viewpoint covers one or more stakeholder concerns** (e.g., security, deployment, ...) While most viewpoints are graphical they do not need to be (e.g., textual viewpoints).
- **Architectural view: Concrete representation of a system through a viewpoint.** Often, viewpoints are left implicit, or represented through a legend.

Collection of software artefacts: More than just source code



Collection of software artefacts: A concrete example with Python on GitHub



- Running example: **Official Google Cloud Spanner Dialect for Hibernate ORM**
- <https://github.com/GoogleCloudPlatform/google-cloud-spanner-hibernate>
- “Init” in 2019
- 35 contributors
- 715 commits (~one commit every three days)
- 30 releases
- Latest release 3 weeks ago (3.3.0)



Collection of software artefacts: A concrete example with Python on GitHub

The screenshot shows the GitHub repository page for `google-cloud-spanner-hibernate`. The repository is public and has 14 watchers, 35 forks, and 47 stars. The main content area displays a list of files and folders, including `.cloudbuild`, `.github`, `.kokoro`, `.mvn/wrapper`, `checkstyle`, `google-cloud-spanner-hibernate-dialect`, `google-cloud-spanner-hibernate-samples`, `google-cloud-spanner-hibernate-testing`, `google-cloud-spanner-hibernate-tools`, `.gitignore`, `CHANGELOG.md`, `CONTRIBUTING.md`, `LICENSE`, `README.adoc`, `mvnw`, and `mvnw.cmd`. The right sidebar contains information about the repository, including the README, license (LGPL-2.1), activity, custom properties, and releases. The latest release is `v3.3.0`, published 3 weeks ago. The contributors section shows 23 contributors.

GoogleCloudPlatform / google-cloud-spanner-hibernate

Code Issues (12) Pull requests (10) Actions Projects Wiki Security Insights

google-cloud-spanner-hibernate (Public)

Watch (14) Fork (35) Star (47)

master 23 Branches 30 Tags

Go to file Add file Code

dependabot[bot] deps: bump org.apache.maven.plugins:maven-gpg-plug... 714d116 · 5 days ago 715 Commits

.cloudbuild	Split tests into nightly and presubmit (#336)	3 years ago
.github	chore(build): make integration tests conditional on secret...	last month
.kokoro	build: add workaround for nexus plugin (#830)	5 months ago
.mvn/wrapper	chore: delete .mvn/wrapper/maven-wrapper.jar (#493)	2 years ago
checkstyle	test: add mock server tests for schema generation (#686)	7 months ago
google-cloud-spanner-hibernate-dialect	chore(master): release 3.3.1-SNAPSHOT (#986)	3 weeks ago
google-cloud-spanner-hibernate-samples	chore(master): release 3.3.1-SNAPSHOT (#986)	3 weeks ago
google-cloud-spanner-hibernate-testing	chore(master): release 3.3.1-SNAPSHOT (#986)	3 weeks ago
google-cloud-spanner-hibernate-tools	chore(master): release 3.3.1-SNAPSHOT (#986)	3 weeks ago
.gitignore	fix: Close session and sessionFactory when finished (#218)	4 years ago
CHANGELOG.md	chore(master): release 3.3.0 (#965)	3 weeks ago
CONTRIBUTING.md	docs: Add contribution guidelines (#990)	2 weeks ago
LICENSE	Initial commit	6 years ago
README.adoc	docs: add a direct reference to the sample application (#...	3 weeks ago
mvnw	Add Sonar Cloud Integration to Travis CI (#26)	5 years ago
mvnw.cmd	Add Sonar Cloud Integration to Travis CI (#26)	5 years ago

About

The official Google Cloud Spanner Dialect for Hibernate ORM

Readme

LGPL-2.1 license

Activity

Custom properties

47 stars

14 watching

35 forks

Report repository

Releases (30)

v3.3.0 (Latest) 3 weeks ago

+ 29 releases

Packages

No packages published

Contributors (23)

+ 9 contributors

Having a glance at the web interface is a good place to start, but can I efficiently mine artefacts this way?

Collection of software artefacts: A concrete example with Python on GitHub

- **Basic example** on conducting repository analysis
- Tools used:
 - a. **GitHub**
 - b. **Python 3** with **PyDriller**
 - c. Basic **shell commands**
- **Overall intuition:**
 - a. Interface with GitHub through a Python Script
 - b. Collect some quick historical data (e.g., commit frequency)
 - c. Run a small example of “more advanced” analysis locally



Collection of software artefacts: A concrete example with Python on GitHub



Interface with GitHub through a Python script: Looping through commits

```
# Import mining library
from pydriller import Repository

# Traverse through each commit in the repository
for commit in
Repository('https://github.com/GoogleCloudPlatform/google-cloud-spanner-hibernate4').traverse_commits():

    # Print commit metadata
    print('Commit hash:', commit.hash)
    print('Commit message:', commit.msg)
    print('Commit author:', commit.author.name)

# Print modified files in commit
for file in commit.modified_files:
    print(file.filename, ' has changed')
```

Interface with GitHub through a Python script: Looping through commits

```
~ > /opt/homebrew/bin/python3.9mining_tutorial.py
17s
Commit hash: adbbd10ae382ecc76b4f86937048c93ef1d7a364
Commit message: Initial commit
Commit author: Mike Eltsufin
CONTRIBUTING.md  has changed
LICENSE  has changed
README.md  has changed
...
```

Which insight on software development activities can I understand from this data?

What interesting
information did I miss
mining?

Looping through commits, and starting to get the best out of it (**history: cumulative commits**)

```
# Import libraries
from pydriller import Repository
from collections import defaultdict
from datetime import datetime

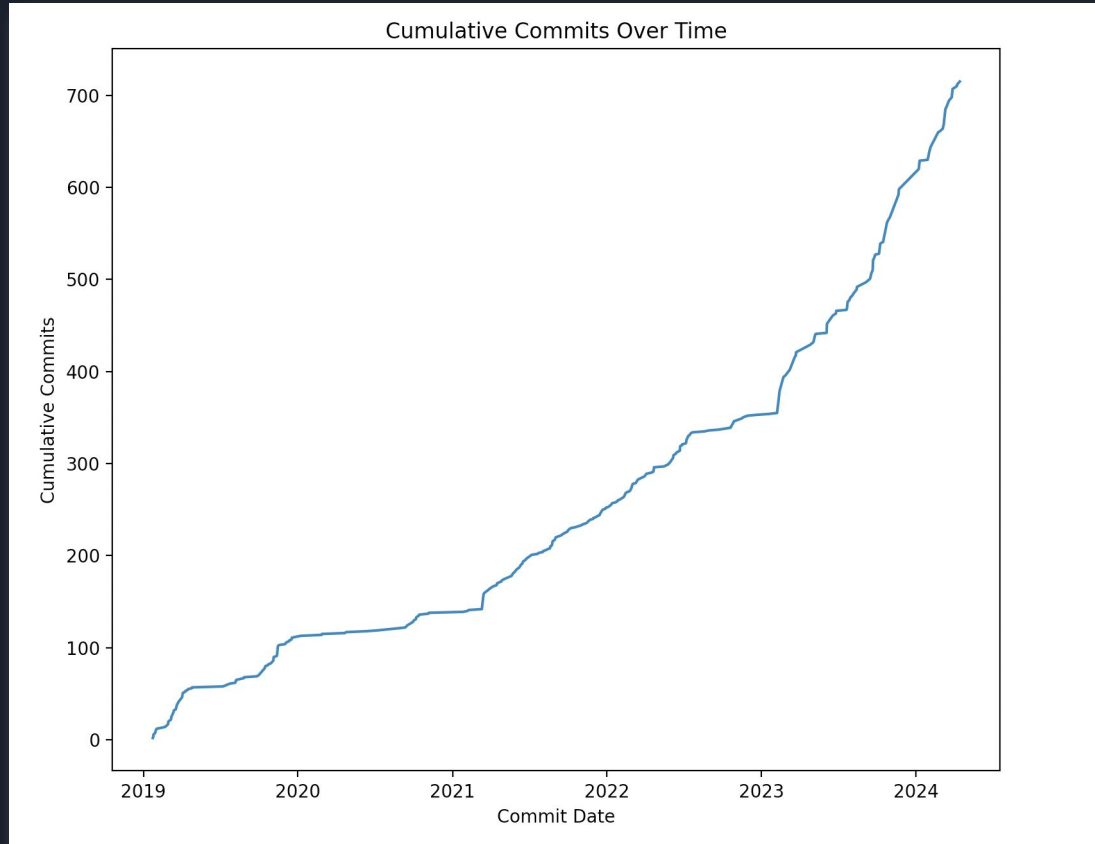
# Initialize a dictionary to store commit dates
commit_dates = defaultdict(int)

# Traverse through each commit in the repository
for commit in Repository('https://github.com/GoogleCloudPlatform/google-cloud-spanner-hibernate').traverse_commits():
    # Store the commit date
    commit_date = commit.committer_date.date()
    commit_dates[commit_date] += 1

# Sort the dates and calculate the cumulative commits
dates = sorted(commit_dates.keys())
cumulative_commits = [sum(commit_dates[date] for date in dates if date <= current_date) for current_date in dates]
```

Why cumulative commits?

Looping through commits, and starting to get the best out of it (history: cumulative commits)



Could it be interesting to
see the number of commits
per day?

Looping through commits, and starting to get the best out of it (history: commits per day)

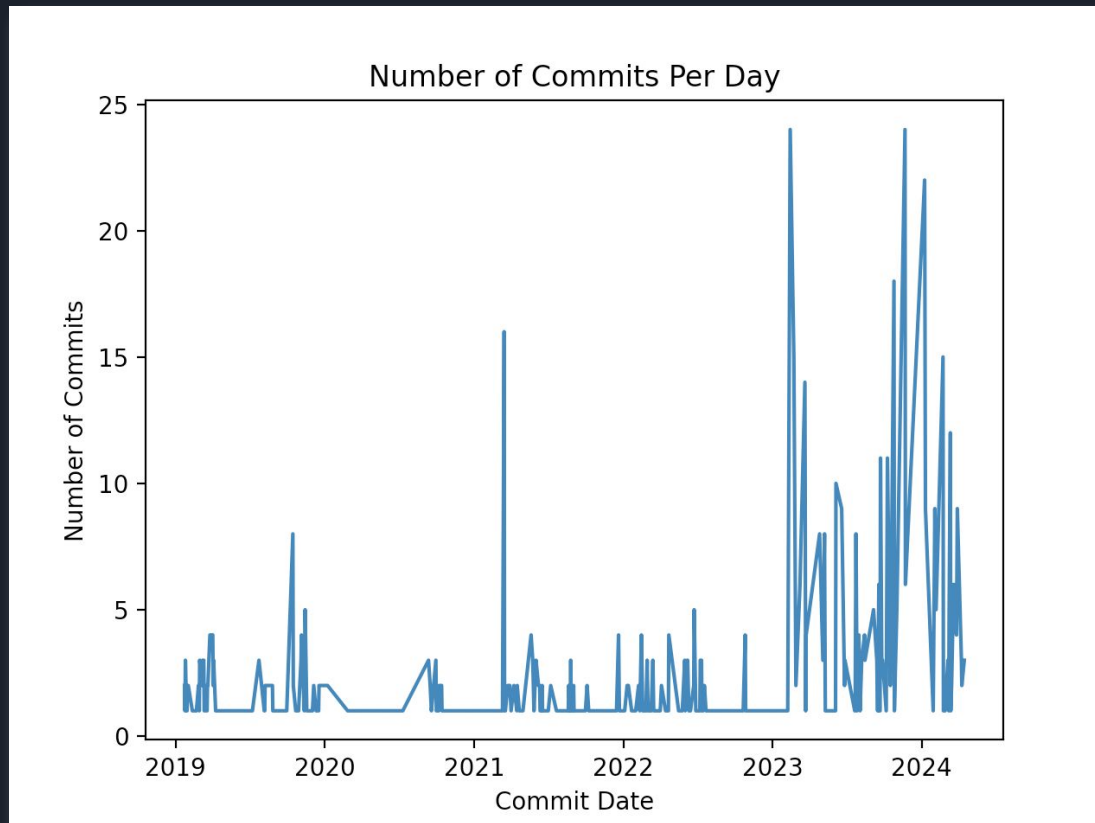
```
# Import libraries
from pydriller import Repository
from collections import defaultdict
from datetime import datetime

# Initialize a dictionary to store commit dates
commit_dates = defaultdict(int)

# Traverse through each commit in the repository
for commit in Repository('https://github.com/GoogleCloudPlatform/google-cloud-spanner-hibernate').traverse_commits():
    # Store the commit date
    commit_date = commit.committer_date.date()
    commit_dates[commit_date] += 1

# Sort the dates and get the number of commits per day
dates = sorted(commit_dates.keys())
commits_per_day = [commit_dates[date] for date in dates]
```

Looping through commits, and starting to get the best out of it (history: commits per day)



I want more insights on this system.
Which file should I look at first?
Who should I talk to?

Looping through commits, and starting to get the best out of it (history: **most frequent committer and changed file**)

```
# Import libraries
from pydriller import Repository
from collections import Counter, defaultdict

# Initialize counters for authors and files
author_counter = Counter()
file_counter = Counter()

# Traverse through each commit in the repository
for commit in
Repository('https://github.com/GoogleCloudPlatform/google-cloud-spanner-hibernate').traverse_commits():
    # Count the authors
    author_counter[commit.author.name] += 1

    # Count the changed files
    for file in commit.modified_files:
        file_counter[file.filename] += 1

# Find the most active author and most changed file
most_active_author = author_counter.most_common(1)
most_changed_file = file_counter.most_common(1)
```

Looping through commits, and starting to get the best out of it (history: most frequent committer and changed file)

```
~ > /opt/homebrew/bin/python3.9 mining_tutorial.py
```

```
17s
```

```
Most active author: [('dependabot[bot]', 442)]
```

```
Most changed file: [('pom.xml', 987)]
```

I need a new strategy, what
should I do?

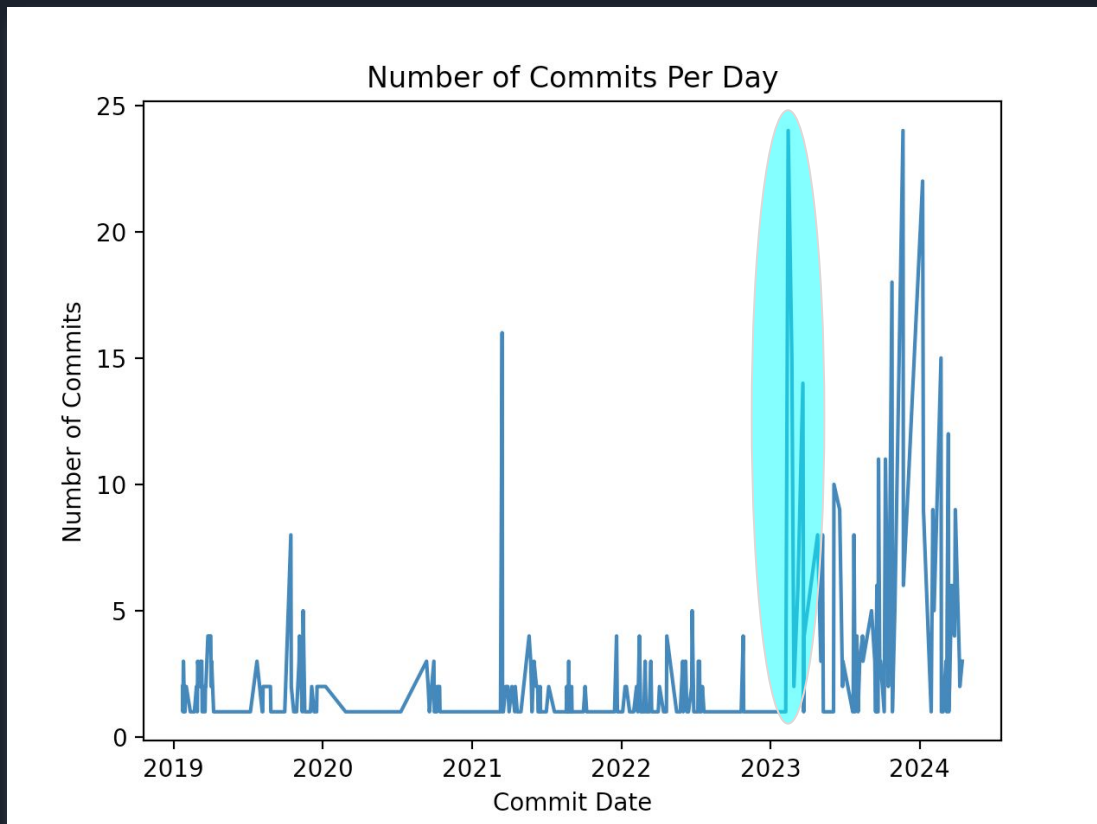
Looping through commits, and starting to get the best out of it (history: most frequent committer and changed file)

```
~ > /opt/homebrew/bin/python3.9 mining_tutorial.py
```

```
22s
```

```
Most active author: [('dependabot[bot]', 442), ('Daniel Zou', 70), ('Knut Olav Løite', 46),  
('release-please[bot]', 40), ('Mike Eltsufin', 38), ('Chengyuan Zhao', 32), ('Deepankar Dixit', 14),  
('dependabot-preview[bot]', 10), ('Ray Tsang', 4), ('Elena Felder', 4)]  
Most changed file: [('pom.xml', 987), ('README.adoc', 72), ('versions.txt', 41), ('SpannerDialect.java',  
31), ('CHANGELOG.md', 22), ('SpannerTableExporterTests.java', 21), ('Person.java', 21), ('Singer.java',  
20), ('SpannerTableExporter.java', 19), ('GeneratedCreateTableStatementsTests.java', 19)]
```

Looping through commits, and starting to get the best out of it (history: Doomsday 🔥?)



Looping through commits, and starting to get the best out of it (history: Doomsday 🔥?)

Pseudocode to identify most frequent committer on “doomsday”:

1. Initialize a dictionary to store commit dates and details
2. For each commit in the repository:
 - 2.1 Store the commit date, author, and message
3. Find the day with the most commits
4. Get the authors and messages of the commits on that day
5. Count all author's commits
6. Find the author who committed the most
7. Get the commit messages of the most active author, excluding Dependabot
8. Store the day with the most commits, the authors active that day, and the files changed that day

Note: This does not consider how ties are resolved (easy way out, consider ties as equally important, if too many, use a sampling strategy)

Looping through commits, and starting to get the best out of it (history: Doomsday 🔥?)

```
~ > /opt/homebrew/bin/python3.9 mining_tutorial.py
```

```
25s
```

```
The most active day was: 2023-02-13
```

```
The most active author on this day was: [('dependabot[bot]', 24)]
```

```
The most changed file on this day was: [('pom.xml', 13)]
```

Looping through commits, and starting to get the best out of it (history: Doomsday 🔥?)

Pseudocode to identify most frequent committer on “doomsday”:

1. Initialize a dictionary to store commit dates and details
2. For each commit in the repository:
 - 2.1 Store the commit date, author, and message
3. Find the day with the most commits
4. Get the authors and messages of the commits on that day
5. Count all author's commits, **excluding “dependabot[bot]”**
6. Find the author who committed the most
7. Get the commit messages of the most active author
8. Store the day with the most commits, the authors active that day, and the files changed that day

Looping through commits, and starting to get the best out of it (history: Doomsday 🔥?)

```
~ > /opt/homebrew/bin/python3.9 mining_tutorial.py
```

```
27s
```

```
The most active day was: 2021-03-15
```

```
The most active author on this day was: [('dependabot-preview[bot]', 9)]
```

```
The most changed file on this day was: [('pom.xml', 10)]
```

Looping through commits, and starting to get the best out of it (history: Doomsday 🔥?)

Pseudocode to identify most frequent committer on “doomsday”:

1. Initialize a dictionary to store commit dates and details
2. For each commit in the repository:
 - 2.1 Store the commit date, author, and message
3. Find the day with the most commits
4. Get the authors and messages of the commits on that day
5. Count all author's commits, **excluding all users ending in “[bot]”** 🤖
6. Find the author who committed the most
7. Get the commit messages of the most active author
8. Store the day with the most commits, the authors active that day, and the files changed that day

Looping through commits, and starting to get the best out of it (history: **No Doomsday!** 🐳)

```
~ > /opt/homebrew/bin/python3.9 mining_tutorial.py
```

```
25s
```

```
The most active day was: 2019-10-15
```

```
All active authors on this day were: {'Mike Eltsufin': 8}
```

```
All files changed on this day were: {'README.adoc': 1, 'pom.xml': 11, 'common.cfg': 1,  
'continuous.cfg': 1, 'drop.cfg': 1, 'drop.sh': 1, 'promote.cfg': 1, 'promote.sh': 1,  
'stage.cfg': 1, 'stage.sh': 2}
```

Can I identify number of bugs through GitHub metadata? How can I measure it?

More “advanced” investigations: Local analyses

- **GitHub metadata** provides interesting information, but is **semantically limited**
- More advanced source code analysis require to:
 - Locally **check-out a commit**
 - **Run analysis locally**
 - Store results
 - **Repeat**
- Pros:
 - Ability to run **whichever analysis you want**
- Cons:
 - Requires **local set-up**
 - Can be more **time consuming**
 - Can be **resource intensive**
 - May require **compiled code or code execution**



Local Analyses: A concrete example with PMD



- Example tool: <https://pmd.github.io>
- PMD: Cross-language **static code analyzer**
- Based on a **set of rules**, e.g., for Java
 - ~120 rules
 - **Covered categories:** best practices, code style, design, documentation, error prone, multithreading, performance, security
 - Rules have **different levels of implementation complexity**, and may have **different priority, weights and thresholds**
- All **rule information** are usually **documented**
- **Rules can be customized** (e.g., custom thresholds and add remove rules from analysis)



Local Analyses: A concrete example with PMD



- Example of simple rule:
 - Rule name: CommentContent
 - Since: PMD 5.0
 - Priority: Medium (3)
 - Short definition: A rule for the politically correct... we don't want to offend anyone.
 - Example:

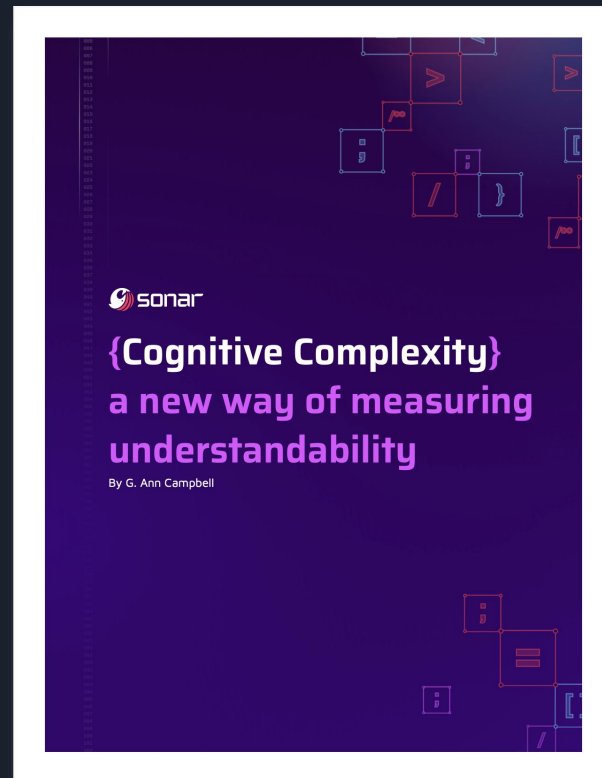
```
//OMG, this is horrible, Bob  
is an idiot !!!
```



Local Analyses: A concrete example with PMD



- Example of complex rule:
 - Rule name: CognitiveComplexity
 - First developed in 2021 by G. Ann Campbell (Sonar)
 - Short definition: Measure of how difficult it is for humans to read and understand a method
 - Intuition: Analyze control flow of methods according to different criteria (e.g., conditional statements, logic operators, jumps, etc.)
 - 22 page documentation:
<https://www.sonarsource.com/docs/CognitiveComplexity.pdf>
 - Default Threshold set to “15”



Local Analyses: A concrete example with PMD

```
# PART 1

# Import libraries
from pydriller import Repository
import os

# Specify the repository
repo_url = "https://github.com/GoogleCloudPlatform/google-cloud-spanner-hibernate"
repo_folder = "msr_temp"

# Clone the repository
clone_command = f'git clone {repo_url} {repo_folder}'
os.system(clone_command)

# Initialize the Repository folder
repo = Repository(repo_folder)
```

Local Analyses: A concrete example with PMD

```
# PART 2

# Traverse through each commit in the repository
for commit in repo.traverse_commits():
    # Checkout each commit
    checkout_command = f'git -C {repo_folder} checkout {commit.hash}'
    os.system(checkout_command)

    # Define shell command to launch PMD
    pmd_cmd =
        f"pmd -f csv -R /Users/roberto/Documents/GitHub/TLTC-tool/ruleset.xml -d {repo_folder}
        -r temp.csv"

    print("Calculating cognitive complexity...")

    # Execute the command
    os.system(pmd_cmd)
```

Local Analyses: A concrete example with PMD

```
~ > /opt/homebrew/bin/python3.9 mining_tutorial.py
```

```
10m 58s
```

```
Cloning into 'msr_temp'...
```

```
remote: Enumerating objects: 7380, done.
```

```
...
```

```
Note: switching to 'adbbd10ae382ecc76b4f86937048c93ef1d7a364'.
```

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

```
...
```

```
HEAD is now at adbbd10 Initial commit
```

```
Calculating cognitive complexity...
```

```
Previous HEAD position was adbbd10 Initial commit
```

```
HEAD is now at f426018 Basic project structure (#1)
```

```
Calculating cognitive complexity...
```

```
...
```

A quick note: Selecting repositories

- **Not every repository** might be suited for the study
- **Specialized criteria** depend on the investigation and could be type of architecture (e.g., microservices), development stack (e.g., Apache Tomcat) presence of certain artefacts (e.g., tests), ...
- **General “quality” criteria** usually are:
 - **Language** (Study dependent: Heterogeneous? Homogeneous?)
 - **Number of stars** (e.g., at least 20)
 - **Number of contributors** (e.g., at least 5)
 - **Number of commits** (e.g., at least 200)
 - **Being active** (e.g., being updated less than 2 months ago)
 - **Size** (e.g., at least having 1k SLOC)
 - **Not being a demo, tutorial, classwork**, etc.
- A good place to start could be <https://seart-ghs.si.usi.ch/>
- Number of repositories can range greatly (e.g., 20 or 200k)

The screenshot displays the search interface of the seart-ghs.si.usi.ch tool. The interface is organized into several sections for filtering repositories:

- General:** Includes a search bar with a dropdown menu (set to 'Contains'), a 'Language' filter, and checkboxes for 'License' and 'Has topic'.
- History and Activity:** Features filters for 'Number of Commits', 'Number of Contributors', 'Number of Issues', 'Number of Pull Requests', 'Number of Branches', and 'Number of Releases'. Each filter has 'min' and 'max' input fields.
- Date-based Filters:** Includes 'Created Between' and 'Last Commit Between', both with date pickers (dd/mm).
- Popularity Filters:** Includes 'Number of Stars' with 'min' and 'max' input fields.
- Size of codebase:** Includes 'Non Blank Lines', 'Code Lines', and 'Comment Lines', each with 'min' and 'max' input fields.
- Additional Filters:** Includes a 'Sorting' dropdown (set to 'Name') and a 'Repository Characteristics' section with checkboxes for 'Exclude Forks', 'Only Forks', 'Has Wiki', 'Has License', 'Has Open Issues', and 'Has Pull Requests'.

A 'Search' button is located at the bottom right of the interface.

Local Analyses: A concrete example with PMD

temp				
Filter				
File	Priority	Line	Description	
msr_temp/google-cloud-spanner-hibernate-tools/src/test/java/com/google/cloud/spanner/hibernate/PooledBitReversedSequenceStyleGeneratorTest.java	3	193	The method 'asProperties(Map)' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-tools/src/main/java/com/google/cloud/spanner/hibernate/TransactionTagInterceptor.java	3	70	The constructor 'TransactionTagInterceptor(Set, boolean)' has a cognitive complexity of 2, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-tools/src/main/java/com/google/cloud/spanner/hibernate/PooledBitReversedSequenceStyleGenerator.java	3	136	The method 'determineSequenceName(JdbcEnvironment, Properties)' has a cognitive complexity of 3, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-tools/src/main/java/com/google/cloud/spanner/hibernate/BitReversedSequenceStructure.java	3	70	The method 'buildSkipRangeOptions(List)' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-tools/src/main/java/com/google/cloud/spanner/hibernate/AbstractTransactionTagInterceptor.java	3	38	The constructor 'AbstractTransactionTagInterceptor()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-testing/src/test/java/org/hibernate/jpa/test/BaseEntityManagerFunctionalTestCase.java	3	190	The method 'buildSettings()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-testing/src/test/java/org/hibernate/jpa/test/BaseCoreFunctionalTestCase.java	3	112	The method 'buildSessionFactory(Consumer)' has a cognitive complexity of 2, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-testing/src/test/java/org/hibernate/dialect/InterleavedTest.java	3	98	The method 'verifyInterleavedTablesDdl()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-testing/src/test/java/org/hibernate/dialect/ForeignKeyTest.java	3	184	The method 'verifyOnDeleteCascadeReferentialConstraints()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-testing/src/test/java/org/hibernate/dialect/entities/JsonEntity.java	3	86	The method 'equals(Object)' has a cognitive complexity of 3, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-sample/src/test/java/com/example/CoffeeApplicationIT.java	3	64	The method 'testCreateCustomer()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-sample/src/main/java/com/example/WebController.java	3	75	The method 'orderCoffee(String, String, int)' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/test/java/com/google/cloud/spanner/sample/SampleApplicationEmulatorTest.java	3	50	The method 'cleanup()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/VenueService.java	3	68	The method 'generateRandomVenues(int)' has a cognitive complexity of 4, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/TrackService.java	3	60	The method 'generateRandomTracks(int, int)' has a cognitive complexity of 3, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/TicketSaleService.java	3	69	The method 'generateRandomTicketSales(int)' has a cognitive complexity of 4, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/StateReadService.java	3	44	The method 'getCurrentTimestamp()' has a cognitive complexity of 2, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/SingerService.java	3	70	The method 'printSingersWithLastNameStartingWith(String)' has a cognitive complexity of 7, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/RandomDataService.java	3	241	The method 'getRandomNickNames()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/ConcertService.java	3	77	The method 'generateRandomConcerts(int)' has a cognitive complexity of 3, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/service/AlbumService.java	3	69	The method 'generateRandomAlbums(int)' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/SampleApplication.java	3	136	The method 'printData()' has a cognitive complexity of 1, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spring-data-jpa-full-sample/src/main/java/com/google/cloud/spanner/sample/entities/Track.java	3	87	The method 'equals(Object)' has a cognitive complexity of 2, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spanner-hibernate-codelab/src/main/java/codelab/Singer.java	3	94	The method 'equals(Object)' has a cognitive complexity of 7, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spanner-hibernate-codelab/src/main/java/codelab/App.java	3	85	The method 'readData(Session)' has a cognitive complexity of 2, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/spanner-hibernate-codelab/src/main/java/codelab/Album.java	3	78	The method 'equals(Object)' has a cognitive complexity of 5, current threshold is 1	
msr_temp/google-cloud-spanner-hibernate-samples/basic-spanner-features-sample/src/main/java/com/example/TransactionTypeDemo.java	3	49	The method 'runReadOnlyTransaction(SessionHelper)' has a cognitive complexity of 3, current threshold is 1	
Showing 1 to 26 of 216 entries, 8 total columns				

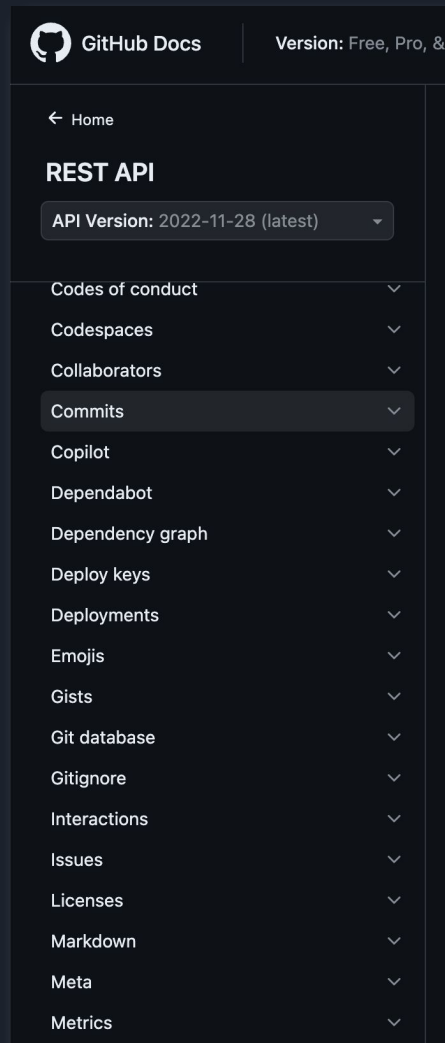
What analyses could I do
with the data I just
collected?

How could I combine commit
history analysis and the
cognitive complexity one?

How would you *locally*
identify number of bugs
evolution trends?

One Further Information: GitHub API

- **Different way** to **interact with GitHub repositories**
- GitHub provides a **REST API** from which information can be mined
- **Numerous API endpoints**, including: Commit info, Issues, Repository content, Search Code, ...
- **Very powerful** for **collecting targeted information** from **as many repositories as possible**
- Pro: Much more **scalable** than checking-out repositories
- Con: Can only be used to **analyze repository “metadata”** (you cannot run source code analysis)
- **Limits: 5,000 requests per hour** for authenticated users
- Very **extensive documentation**:
<https://docs.github.com/en/rest>



One Further Information: GitHub API

```
# Locate all files on GitHub containing the string  
"pangoccioli"
```

```
curl -L -H "Accept: application/vnd.github+json"
```

```
-H "Authorization: Bearer <YOUR TOKEN>"
```

```
-H "X-GitHub-API-Version: 2022-11-28"
```

```
-H "https://api.github.com/search/code?q=pangoccioli"
```

One Further Information: GitHub API

```
~ > curl -L -H "Accept: application/vnd.github+json" -H "Authorization: Bearer <YOUR TOKEN>" -H
"X-GitHub-API-Version: 2022-11-28" -H "https://api.github.com/search/code?q=pangoccioli"
0.01s
{
  "total_count": 10,
  "incomplete_results": false,
  "items": [
    {
      "name": "Program.cs",
      "path": "04_EserciziProgrammazioneConcorrente/EsercizioPangoccioli/Program.cs",
      "sha": "8d12358388d4775410f85cfbd1c2b28968e560d9",
      "url":
"https://api.github.com/repositories/587688977/contents/04_EserciziProgrammazioneConcorrente/EsercizioPangoccioli/Program.cs?
ref=2dec6e0345b61e6a57da9635ca9677a5f0bffb33",
      "git_url": "https://api.github.com/repositories/587688977/git/blobs/8d12358388d4775410f85cfbd1c2b28968e560d9",
      "html_url":
"https://github.com/GiorgioCitterio/InformaticaScuola/blob/2dec6e0345b61e6a57da9635ca9677a5f0bffb33/04_EserciziProgrammazione
Concorrente/EsercizioPangoccioli/Program.cs",
      "repository": {
        "id": 587688977,
```

One Further Information: GitHub API

```
# Locate all README files on GitHub containing the string "hack"
```

```
curl -L -H "Accept: application/vnd.github+json"
```

```
-H "Authorization: Bearer <YOUR TOKEN>"
```

```
-H "X-GitHub-API-Version: 2022-11-28"
```

```
-H "https://api.github.com/search/code?q=hack+in:file+filename:README.md"
```

One Further Information: GitHub API

```
~ > curl -L -H "Accept: application/vnd.github+json" -H "Authorization: Bearer <YOUR TOKEN>" -H
"X-GitHub-API-Version: 2022-11-28" -H "https://api.github.com/search/code?q=hack+in:file+filename:README.md"
0.02s
{
  "total_count": 13200,
  "incomplete_results": false,
  "items": [
    {
      "name": "README.md",
      "path": "hack/README.md",
      "sha": "6afd08d9f497b59e344488df37c2cba907e58661",
      "url":
"https://api.github.com/repositories/20580498/contents/hack/README.md?ref=5a71f37805246db1f7f681c4ee7da508054cc429",
      "git_url": "https://api.github.com/repositories/20580498/git/blobs/6afd08d9f497b59e344488df37c2cba907e58661",
      "html_url": "https://github.com/kubernetes/kubernetes/blob/5a71f37805246db1f7f681c4ee7da508054cc429/hack/README.md",
      "repository": {
        "id": 20580498,
        "node_id": "MDEwOlJlcG9zaXRvcnkYMDU4MDQ5OA==",
        "name": "kubernetes",
        "full_name": "kubernetes/kubernetes",
```

Can I conduct a “case study”
with the GitHub API?

What other type of studies
can I do?

Two words on the final project: Overall structure

- **Quantitative or qualitative** inquiry on a **software engineering topic**
- Can be conducted in **teams** (max. 3 people)
- **Weekly joint “stand-ups”** to present progress, discuss impediments, ...
- Final “exam”:
 - **Structured deliverable** documenting the analysis and the results, accompanied by a “replication package”, and
 - **Hold a presentation** of the results
- Project can naturally develop in a thesis subject
- High quality deliverables could be reworked into a scientific publication



Two words on the final project: Regarding the topic of choice

- You must choose a topic that **interests you**
- The topic should have a **practical motivation** (e.g., no *“On the Evolution of the “hack” Keyword in README Files”*)
- **Read some papers before jumping into the project.**
Be creative, be curious, be smart
- If you have a loose idea, the **topics can jointly finalized with me**
- If clueless, **I can provide ideas**
- The final topic choice is yours. However I can support you most on topics related to **software quality, software architecture, green software, technical debt, testing, software education**
- **“Everything” goes:** StackOverflow, ChatGPT, chatting with your lead architect friend, call at home, etc.
However, you have to know what you are doing, and not commit plagiarism.
- Contact me immediately if “lifting” instances develop (not at the end of the project)



*Final word on the project: Above and beyond,
the sky is (not?) the limit*

You

1337 h4x0r! It worked magic!

Once my lecturer at my 2nd undergraduate year said:

"There are two types of people: normal ones and wizards. You are supposed to be the wizards" 🧙

11:46

Next week

A Deep-dive into ~~Pokemon~~ Data Analysis

- Please make sure to:
 - **Bring you laptop**
 - **Have RStudio installed on it**



Do you have a question or doubt? Reach out!

ROBERTO VERDECCHIA CURRICULUM PUBLICATIONS RESEARCH TEACHING SERVICES AWARDS MISC





Roberto Verdecchia
*Computer Scientist,
Assistant Professor at
UniFI.*

Hi there! I am **Roberto Verdecchia**, an Assistant Professor at the [STLAB](#) of the [University of Florence](#).

My research interest lies in the intersection between *Software Architecture* and *Technical Debt* (which combined can look something like [this](#)). I am also fascinated by topics related to *Software Sustainability*, *Software Testing*, and *Software Education*.

Prior to joining the [STLAB](#), I was a Postdoctoral Research Associate of the [S2Group](#) at the [Vrije Universiteit Amsterdam](#).

I hold a Double Ph.D. obtained at the [Computer Science Department](#) of the [Gran Sasso Science Institute](#) and at the [S2Group](#) of the [Vrije](#)



roberto.verdecchia@unifi.it



SCAN ME

<https://robertoverdecchia.github.io>

Oh look! We still have heaps of time!

Let's make the best out of it!

- **Option 1**

Mining studies are hard!
Let's look at one together!

https://robertoverdecchia.github.io/papers/MSR_2020.pdf

- **Option 2**

Got all the basics of mining studies?
Let's design a study together!